



UNIWERSYTET VIZJA

Zaawansowane zagadnienia algorytmiki i programowania Projekt

Algorytmy w teorii liczb i kryptografia
Projekt dotyczy implementacji algorytmów arytmetyki modularnej
i teorii liczb w kontekście kryptografii (RSA)

Prowadzący

dr hab. inż. Agnieszka Duraj

Studenci

Jakub Peciak Album nr: 65848

Paweł Cabaj Album nr 65456

Warszawa 2025

Raport 3

Rozszerzenia i optymalizacja

Spis treści

1. Kluczowe zagadnienia wstępne
2. Implementacja dodatkowych metod
3. Analiza złożoności
 - A. Dla metody próbnych podziałów
 - B. Dla metody z testem Millera – Rabina
 - C. Zestawienie kosztów
4. Ocena różnic obu wersji
5. Poprawność i spójność kodu

1. Kluczowe zagadnienia wstępne

Rozszerzenie i optymalizacja implementacji do potrzeb testów w projekcie ogranicza się do przedstawienia dwóch sposobów generowania liczb pierwszych. Ich działanie jest tożsame co do sposobu zastosowania. Natomiast różnią się znacząco co do czasu wykonania - kosztu algorytmu.

Pierwszy z nich jest zdecydowanie wolny i przeznaczony do celów testowych z użyciem krótkich kluczy. Jednocześnie wskazuje jak ważne w całym algorytmie jest generowanie liczb pierwszych, gdyż jak się okazuje jest kluczowe dla wyznaczania klucza RSA. A ściślej czasu jego generowania. Poniżej algorytm prosty.

```
# Generuj liczbę pierwszą
def generate_prime_old(min, max, exclude):          # Zakres oraz wartość wykluczona
    while True:
        a = random.randint(min, max)              # Losowanie liczby
        if a == exclude: continue                 # Wykluczona wartość (exclude)
        if a % 2 == 0: continue                   # Odrzucenie parzystych
        for i in range(3, int(a**0.5)+1, 2):       # Poszukiwanie wśród nieparzystych (do sqrt(a))
            if a % i == 0:                         # Jeśli brak reszty - liczba złożona
                break                               # Porzuć
        else:
            return a                               # Zwróć liczbę pierwszą
```

Poniżej sposób generowania klucza RSA przy użyciu testu Millera – Rabina. Szkielet samej funkcji jest podobny, tylko jako jeden z warunków dodajemy warunek Millera -Rabina. Pozwala on na pominięcie badania wszystkich liczb nieparzystych do $\sqrt{a}$ w zamian za wygenerowanie liczby pierwszej z dokładnością określoną jako parametr liczby iteracji w wykonaniu testu.

```
# Generuj liczbę pierwszą - test Rabina-Millera
def generate_prime(min, max, exclude):             # Zakres oraz wartość wykluczona
    while True:
        a = random.randint(min, max)              # Losowanie liczby
        if a == exclude: continue                 # Wykluczona wartość (exclude)
        if a % 2 == 0: continue                   # Odrzucenie parzystych
        if miller_rabin(a, k=40):                 # Test Rabina-Millera
            return a                               # Zwróć liczbę pierwszą
```

Test ten jako test pierwszości jest metodą probabilistyczną. Zatem otrzymujemy rozwiązanie numeryczne, którego dokładność opiera się na liczbie iteracji. Jednak dla parametru ilości powtórzeń ustawionym na $k = 40$, uzyskujemy wystarczające prawdopodobieństwo uniknięcia błędu i jest one pomijalne przy zastosowaniach RSA. Zasada działania testu Millera – Rabina została opisana w poprzednich raportach. W kodzie poniżej zamieszczono niezbędne komentarze odnoszące się do realizacji algorytmu.

```
# Test R-M
def miller_rabin(n, k=40):
    if n < 2:
        return False
    if n in (2, 3):
        return True
    if n % 2 == 0:
        return False

    # rozkład  $n-1 = d \cdot 2^r$ 
    r = 0
    d = n - 1
    while d % 2 == 0:
        d //= 2
        r += 1

    for _ in range(k):
        a = random.randrange(2, n - 2)
        x = pow(a, d, n)

        if x == 1 or x == n - 1:
            continue

        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False

    return True
```

Liczby mniejsze niż 2 pomijamy
Liczby 2 i 3 są pierwsze
Liczby parzyste (poza 2) nie są pierwsze
Inicjacja licznika r
Część nieparzysta - inicjacja
Dopóki d parzyste ..
Dzielenie d bez reszty
Inkrementuj licznik r
Iteracja k razy
Losowanie podstawy a
Potęgowanie modularne
Pierwszy warunek przejścia
Iteracja r-1 razy
Kolejne potęgowania modularne
Warunek przejścia
Wyjście - liczba jest pierwsza
Liczba n jest złożona - zwróć False
Liczba n jest pierwsza - zwróć True

2. Implementacja dodatkowych metod

W programie wewnątrz ciała zaimplementowano stosowny kod pozwalający na badanie kluczowych parametrów związanych z działaniem algorytmu i generowaniem klucza RSA. Do potrzeb tych testów wykorzystano biblioteki: <time> oraz <psutil>, które to umożliwiają odpowiednio pomiar: czasu wykonywania danych fragmentów kodu oraz druga do pomiaru użycia procesora oraz pamięci RAM komputera, na którym uruchamiamy środowisko projektowe działające na zasadzie serwer - klient. Zgodnie z opisem w poprzednich sprawozdaniach. Analiza pobranych danych znajdzie się w kolejnym raporcie.

```
start = time.perf_counter()           # Odcisk czasu systemowego (start)
cpu_before = process.cpu_times().user  # Inicjacja pomiaru czasu użycia procesora
mem_before = process.memory_info().rss # Inicjacja pomiaru czasu pamięci

... treść badanego kodu

mem_after = process.memory_info().rss  # Odczyt końca pomiaru użycia RAM
cpu_after = process.cpu_times().user   # Odczyt końca pomiaru czasu użycia procesora
stop = time.perf_counter()             # Odcisk czasu systemowego (stop)

czas = stop - start                   # Czas wykonania bloku programu [ms]
cpu_used = cpu_after - cpu_before     # Zmierzony parametr użycia CPU [s]
mem_diff = mem_after - mem_before     # Zmierzony parametr użycia RAM [kB]
```

3. Analiza złożoności

Złożoność obliczeniowa zależy w głównej mierze od złożoności generacji liczb pierwszych. Dla pierwszej metody algorytm składa się z losowania liczb kandydatów oraz sprawdzania pierwszości w kolejnych iteracjach pętli <for> ujętych w pętli powtórzeń <while>. Poniżej przedstawiam złożoność algorytmu bez wyprowadzeń, opierając się na źródłach zewnętrznych.

A. Dla metody próbnych podziałów:

Pojedynczy test to koszt rzędu: $O(\sqrt{n})$

Przeprowadzenie N iteracji: $O(\ln n)$

Całość: $O(\ln n * \sqrt{n})$

B. Dla metody z testem Millera – Rabina mamy następujące koszty:

Pętla główna for: k (stała)

Potęgowanie modularne: $O(\log^3 n)$

Złożoność funkcji okalającej (generate_prime): $O(\ln n)$

Całość: $O(\ln n * \log^3 n)$

C. Zestawienie kosztów

Dla metody próbnych podziałów wzrost obserwacji matematycznej jest wzrostem wykładniczym. Natomiast dla metody z testem Millera – Rabina wzrost ten jest wielomianowy. Oznacza to, iż nawet dla krótkich kluczy, wzrost czasu potrzebnego do obliczeń wraz ze wzrostem długości klucza rośnie dla pierwszego – wykładniczo. Przy większych długościach kluczy każdy dodatkowy bit znacząco wydłuża czas obliczeń. Natomiast dla wersji z testem M – R wzrost ten zachodzi potęgowo (wielomianowo).

4. Ocena różnic obu wersji

Jeśli chodzi o porównanie obu metod uzyskiwania liczb pierwszych, które jest kluczowe przy generacji kluczy RSA, to nieformalnie część tego porównania dokonano już w poprzednich paragrafach.

Po pierwsze porównano clue obu algorytmów. Sam szkielet jest bardzo podobny, jednak w metodzie z testem Millera – Rabina dodajemy tenże test jako jeden z warunków badania pierwszości.

Przedstawiłem już w poprzednim paragrafie porównanie kosztów osiągnięcia wyniku w obu algorytmach. Krótko podsumowując mamy wynik o złożonościach odpowiednio wykładniczych i wielomianowych. To sprawia, że dla nawet stosunkowo małych kluczy nawet tych od 74 bitów, algorytm z testem Millera - Rabina zdecydowanie szybciej liczy liczby pierwsze. A klucze o długościach na przykład 1024 bitów jest przy klasycznym algorytmie praktycznie niemożliwe do policzenia i program blokuje się z powodu przekroczenia systemowych limitów oczekiwania na wynik. Zwyczajnie wynik nieakceptowalny czasowo. Czasy liczenia kluczy i ich analiza będą przedstawione w następnym raporcie.

Porównanie obu algorytmów jest klasycznym porównaniem rozwiązań deterministycznych i probabilistycznych. Istotą jest tu ich skalowalność. Jak już wspomniałem użycie klasycznego algorytmu praktycznie uniemożliwia zastosowanie w praktycznych zastosowaniach poprzez czas generacji i ma właściwie tylko wartość edukacyjną. W metodzie probabilistycznej sterując liczbą iteracji możemy uzyskać odpowiednio duże i pewne liczby pierwsze. Jednocześnie pozwala osiągać kompromis pomiędzy czasem wykonania obliczeń a pewnością warunku pierwszości. Sprawia to, że jest on powszechnie stosowany, jako wystarczający do zachowania warunków bezpiecznego generowania kluczy.

Podsumowując. Przy już stosunkowo niedużych kluczach różnice w czasie generacji liczb pierwszych dla obu algorytmów gwałtownie się zwiększają. Sprawia to, że algorytm deterministyczny zwyczajnie przestaje być użyteczny.

5. Poprawność i spójność kodu

Jakość i rozmiar liczb pierwszych mają bezpośrednie przełożenie na bezpieczeństwo klucza RSA. I jeśli chodzi o dominację czasową to największą część czasu w czasie procesu generowania klucza, jak już wcześniej wspomniano, zajmuje właśnie generowanie bezpiecznych liczb pierwszych. Jakość ta bezpośrednio wpływa na efektywność i bezpieczeństwo.

Implementacje obu algorytmów są zgodne z klasycznymi źródłami i literaturą w tym temacie. Programy serwera i klienta pokazują kolejne etapy od żądania generacji klucza, poprzez testy pierwszości i generację liczb pierwszych, następnie samą generację klucza a także odesłanie go i przesyłanie zakodowanych tym kluczem informacji. Każdy z etapów został ujęty w oddzielnej definicji - zgodnie z budową aplikacji w języku Python.

W ten sposób kod został podzielony na logiczne fragmenty. W algorytmach tych zastosowano warunki spełniania lub kończenia obliczeń stosownie do formalnej struktury metod. Kod został opisany i działa na zasadzie architektury klient - serwer.

Architektura umożliwia przesyłanie danych niekodowanych, generację klucza RSA oraz przesyłanie danych szyfrowanych. Dzięki modularyzacji kod łatwo zbadać. Sama zaś analiza poprawności i spójności potwierdza spełnienie wymagań projektowych od strony programistycznej a także wymagań algorytmicznych.